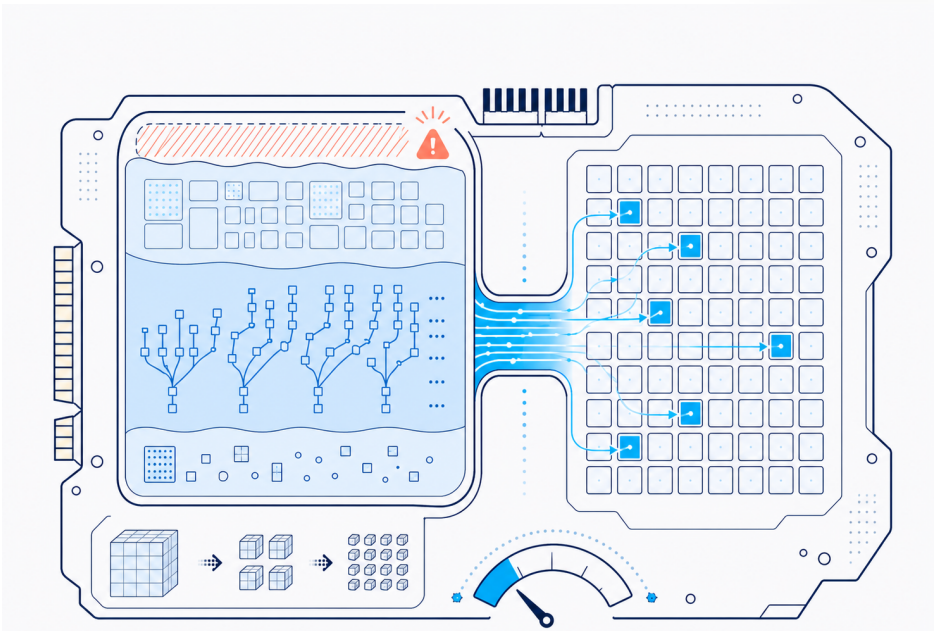
SYSTEMS & HARDWARE

GPU Fundamentals For AI Engineers



You run out of memory long before you run out of math. **TFLOPS, the number everyone quotes, is almost never the thing that limits you.**


AI Coach John FOLLOW | PROITBRIDGE

REPOST

PROITBRIDGE

A GPU Is Memory With Some Math Bolted On

MENTAL MODEL

Not "a fast calculator". A huge fast memory feeding a bank of tiny calculators.

CPU

a few big cores

one hard thing, fast

optimized for latency

GPU

thousands of small cores

one simple thing, a billion times

optimized for throughput

Those thousands of cores are **useless if you cannot feed them.** Every number they multiply has to be read from memory and written back.

A GPU is compute AND a memory system, and they compete. For the 8B model, the card is not compute-starved — **it is memory-starved.**

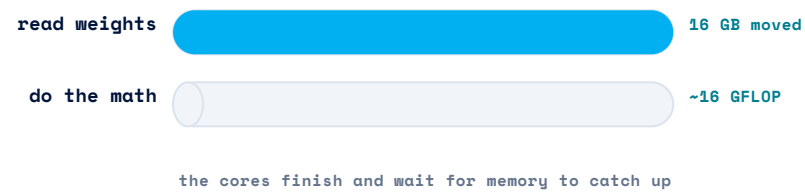
PROITBRIDGE

Inference Is Memory-Bound, Not Compute-Bound

CORRECTION

Generating text does very little math and moves an enormous amount of memory. The cores sit idle.

PER TOKEN · 8B MODEL, FP16



TOKEN RATE IS	set by memory BANDWIDTH (GB/s)
TOKEN RATE IS NOT	set by TFLOPS

A card with **2x the TFLOPS but the same bandwidth** generates tokens at nearly the same speed. If you bought it for the compute number, you paid for what you are not using.

PROITBRIDGE

The VRAM Budget, Where It All Goes

BEFORE SPEED, FIT

Add up what lives in memory at once. The weights are only the first line.

24 GB CARD · 8B MODEL IN FP16



weights	params × bytes — everyone counts this
KV cache	batch × sequence — nobody budgets this
overhead	activations, context — forgotten until OOM

16 GB of weights leaves about **6 to 7 GB** for everything else. That sounds like plenty, until the KV cache starts eating it at long context or high batch.

"An 8B model needs 16 GB" is the answer that gets people **OOM-killed in production.** The weights need 16 GB. The system needs more.

PROITBRIDGE

The Assumption That OOMs You

EXAMPLE

"16 GB model, 24 GB card, plenty of room." Then you raise the batch size and it dies.

THE REASONING ALMOST EVERYONE STARTS WITH

```
# 8B x 2 bytes (fp16) = 16 GB. Card has 24 GB. Fine.
model = load("8b-model", dtype="fp16")   # 16 GB resident

# let's serve 32 users at once for throughput
serve(model, batch_size=32, max_seq_len=8192)   # OOM
```

BATCH	RESIDENT MEMORY	RESULT
batch_size=1	16 GB weights + a small cache	fits
batch_size=8	16 GB weights + a medium cache	tight
batch_size=32	16 GB weights + a LARGE cache	OOM

↕ Nothing about the model got bigger. **The way you served it did.** The KV cache is the memory nobody budgets, and batch size is the lever that spends it.

PROITBRIDGE

The KV Cache Is The Memory Nobody Budgets

HIDDEN COST

To avoid re-reading the conversation every token, the model caches it. That cache is often bigger than the model.

WHAT THE KV CACHE SCALES WITH

```
layers  ×  width  ×  seq_length  ×  batch  ×  bytes
  |          |          |            |
  fixed by the model   grows with   grows with
                       the convo    concurrency
```

ONE 8K USER	roughly a couple of GB of KV cache
THIRTY-TWO OF THEM	dwarfs the 16 GB of weights

Every concurrent user gets their own cache, and it grows as their conversation grows. **Weights are a fixed cost. The KV cache is the variable cost** that scales with traffic.

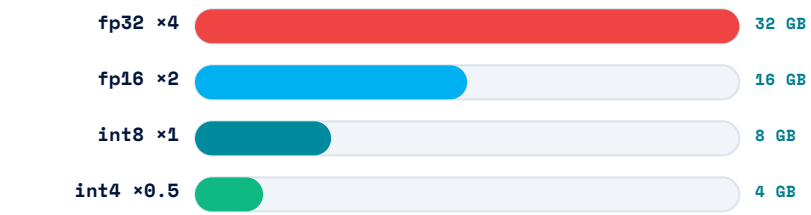
PROITBRIDGE

bytes-per-parameter Is The Number Under Your Control

BIGGEST LEVER

You cannot change how many parameters. You can change how many bytes each one takes.

8B MODEL · WEIGHTS ONLY



fp32 will not fit a 24 GB card · int8 is usually near-lossless · int4 needs measuring

⚠ THE COST

Quantization is not free. **8-bit is usually nearly lossless** and a safe default when VRAM is tight. **4-bit buys a lot of room** and needs you to actually measure whether the output still holds up.

The biggest number in your VRAM budget is one you **get to set.** Params are fixed. Bytes-per-param is your call.

PROITBRIDGE

Batch Size Is Free Throughput, Paid In VRAM And Latency

THROUGHPUT

Bigger batches read the weights once and serve many users with them.

BATCH	THROUGHPUT	VRAM (CACHE)	LATENCY
1	low	tiny	lowest
8	high	fits	acceptable
32	highest*	OOM risk	high

Generation is memory-bound: the expensive step is reading the weights. Batch 32 reads the same 16 GB **once** and makes a token for all 32.

* throughput you cannot allocate is zero. The right batch is **not the biggest one** — it is the one that fits in VRAM AND meets your latency target.

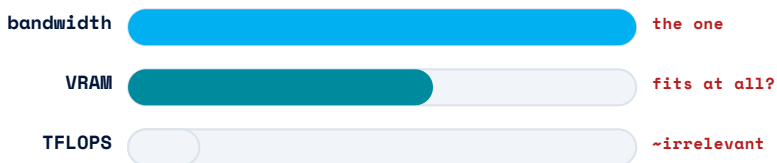
PROITBRIDGE

Everyone Shops For TFLOPS. Bandwidth Is What You Feel.

BLIND SPOT

For inference, the memory bandwidth number predicts your token rate. The compute number mostly does not.

HOW WELL EACH SPEC PREDICTS TOKENS/SEC



read 16 GB at 1 TB/s ≈ 16 ms/token ≈ 60 tok/s, whatever the TFLOPS

Double the bandwidth and you roughly **double the token rate.** Double the TFLOPS at the same bandwidth and you get almost nothing for single-stream generation.

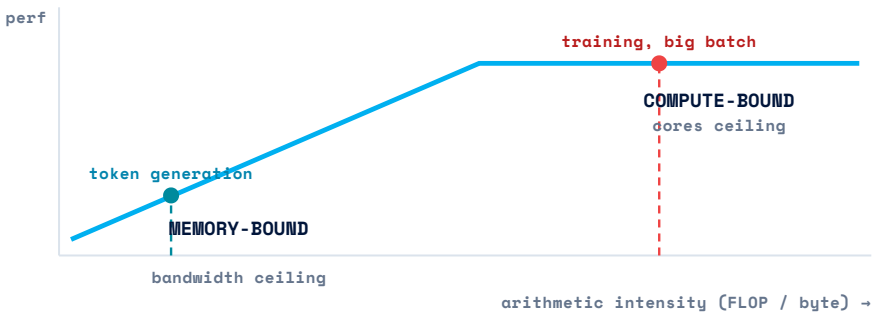
Compare inference cards by **memory bandwidth and VRAM capacity, not TFLOPS.** This is the most useful thing to know when someone hands you a spec sheet.

PROITBRIDGE

The Roofline, The Whole Thing In One Diagram

ONE PICTURE

Every workload is either waiting on memory or waiting on compute. Which one tells you what to fix.



Left of the bend, **memory-bound**: more TFLOPS does nothing — cut bytes moved (quantize, batch). Right of the bend, **compute-bound**: faster memory does nothing — add math throughput.

Token generation sits far left, firmly memory-bound. **Turning the wrong knob is the most common way engineers waste money on hardware.**

PROITBRIDGE

CUDA Cores And Tensor Cores Are Not The Same Engine

TWO ENGINES

The TFLOPS on the box is mostly the Tensor cores. The CUDA cores do everything else.

CUDA CORES

general arithmetic

element-wise ops, the "glue"

modest throughput

run any precision

TENSOR CORES

matrix multiply-accumulate

the big matmuls

the huge TFLOPS number

built for fp16 / bf16 / int8

Transformer inference is dominated by matmuls, which run on **Tensor cores** — and those are fed through lower precision by design.

Forcing fp32 for safety can leave **the fastest hardware on the card idle.** Know which cores your workload uses before you reason about its speed.

PROITBRIDGE

Measure Where The Limit Actually Is

PROVE IT

You do not have to guess memory-bound vs compute-bound. The GPU tells you, if you ask.

WATCH VRAM AND UTILIZATION LIVE

```
nvidia-smi --query-gpu=memory.used,memory.total,\
           utilization.gpu --format=csv -l 1
# high memory.used, moderate util -> memory-bound
```

MEASURE TOKENS/SEC AS YOU SCALE THE BATCH

```
for b in [1, 2, 4, 8, 16, 32]:
    try:
        t = benchmark(model, batch_size=b)  # tok/s, total
        print(f"batch {b}: {t:.0f} tok/s")  # rises, then OOMs
    except OutOfMemoryError:
        print(f"batch {b}: OOM")          # you found the ceiling
        break
```

Utilization below 100% while memory is full = **memory-bound**, fix with fewer bytes. Tokens/sec climbs with batch then cliffs into OOM = your sweet spot and ceiling in one sweep.

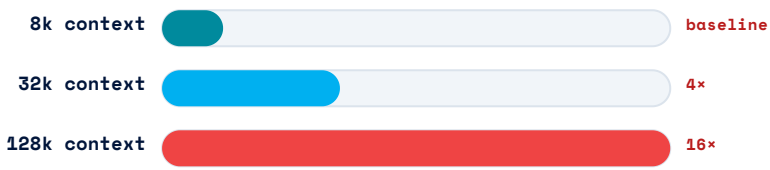
PROITBRIDGE

The Context-Length Trap

THE SILENT ONE

Longer context does not cost more weights. It costs more KV cache, and that grows with every user at once.

KV CACHE PER REQUEST · 8B MODEL · BEFORE MULTIPLYING BY BATCH



a card that served 32 users at 8k might serve 2 at 128k

The weights never changed. The cache did. A card that comfortably served **32 users at 8k** might serve **2 at 128k**, or OOM trying.

The model supported 128k the whole time, but your VRAM budget was written for 8k — and the **first user who fills the window takes down the batch.**

PROITBRIDGE

What Breaks When You Scale Up

SCALING

Single-GPU intuitions hold until the model does not fit on one card. Then new limits appear.

1. **The interconnect becomes the bottleneck.** Cards exchange activations every layer. Over slow PCIe instead of a fast link, communication erases the extra card's benefit.
2. **Memory does not simply add up.** Two 24 GB cards are not a 48 GB card. Splitting has overhead, and some strategies replicate data.
3. **Batches imbalance across cards.** Uneven sequence lengths mean some GPUs finish early and wait, and utilization drops.
4. **More GPUs multiply the failure surface.** A long job is only as reliable as its least reliable card and its link.

Measure the **interconnect bandwidth** the way you measure memory bandwidth on one card. On a multi-GPU box it is often the number that decides throughput.

PROITBRIDGE

Training Flips The Whole Picture

OTHER REGIME

Everything reverses. Training is compute-bound, and its memory is dominated by things inference never allocates.

	INFERENCE	TRAINING
Bound by	memory bandwidth	compute (TFLOPS)
Roofline	far left	right
Memory holds	weights + KV cache	weights + gradients + optimizer + activ
Memory need	~model + cache	several × model

Same card, opposite limits. **"It trained on this GPU" tells you almost nothing about whether it will serve efficiently on it.**

PROITBRIDGE

The Numbers To Carry In Your Head

BACK OF ENVELOPE

A handful of figures that answer most "will it fit, will it be fast" questions on the spot.

QUANTITY	RULE OF THUMB
weights memory	params × bytes/param (fp16=2, int8=1, int4=0.5)
8B in fp16	**~16 GB just for the weights**
KV cache	layers × width × seq × batch; one long seq ≈ GB
inference bound by	**memory bandwidth, not TFLOPS**
training memory	several × the inference memory
token time (rough)	weights_bytes / memory_bandwidth
the batch rule	**biggest batch that FITS and meets latency**

PROITBRIDGE

Fitting A Model On The Card You Have

FIVE LEVERS

Five levers, in the order you should reach for them.

LEVER	WHAT IT COSTS	REACH FOR IT
quantize weights	some quality — measure it	**first, biggest win**
shorten context	less usable window	if unused
reduce batch	lower throughput	if latency allows
offload to CPU/disk	much slower (slow memory)	last resort
add / split GPUs	cost + interconnect limit	nothing else fits

Quantization is almost always the first lever: it shrinks the **biggest line in the budget for the smallest engineering cost.**

PROITBRIDGE

What Is Limiting You Right Now?

DECISION PATH

Three questions locate your bottleneck, and each points at a different fix.

Q1 **Does it fit in VRAM at all?** No → a **capacity** problem. Quantize, shorten context, or split GPUs. More compute will not help.

Q2 **Is GPU utilization low while memory is full?** Yes → **memory-bound**, the normal case. Cut bytes moved: quantize, or batch to amortize weight reads. Faster cores do nothing.

Q3 **Is utilization near 100% and you want it faster?** Yes → **compute-bound** (training, or big-batch). Now TFLOPS matters: more compute, or less work.

Single-stream token generation is almost always the **memory-bound middle branch.** Start your guess there.

What You See, Which Limit Is In Play

DEBUG TABLE

Note how many rows point at memory rather than compute. That is the deck compressed.

SYMPTOM	CAUSE	LIMIT
OOM the moment you raise batch	the KV cache, not the weights	memory
Weights fit but serving crashes	forgot cache + overhead	vram budget
GPU at 40% but memory full	memory-bound generation	bandwidth
2x pricier card, same tok/s	paid for TFLOPS, not bandwidth	wrong metric
Fast at 8k, OOM at 128k	context scales the cache	context
Great on one GPU, slow on four	the interconnect	multi-gpu
Trained fine, serves poorly	training ≠ inference sizing	regime
fp32 slower than expected	Tensor cores left idle	precision
Throughput flat as batch grows	already bandwidth-capped	bandwidth

PROITBRIDGE

You Are Memory-Bound. Start There.

CHEATSHEET

Before you reason about a GPU, ask whether memory or compute is the wall. It is almost always memory.

TOPIC	THE ONE LINE THAT MATTERS
The real limit	VRAM and bandwidth, not TFLOPS, for inference
VRAM budget	weights + KV cache + overhead, not just weights
KV cache	scales with context and batch, often > the model
bytes-per-param	quantization is the biggest lever you control
Batch size	the biggest that FITS and meets your latency
Bandwidth	it sets tokens/sec; buy inference cards by it
Roofline	memory-bound: cut bytes. compute-bound: add math
Training	the opposite regime; never size it by inference
Measure	nvidia-smi and a batch sweep before you buy

Generation is memory-bound. **That one fact reframes every decision on this list.**